# LESSON 12

# 存下来  再来用



## 一维数组入门

学完以后，一串数读完不会丢，想用哪个就用哪个，想用几遍就用几遍。

# 读入 n 个数，倒着输出，怎么办?

lost.cpp（只阅读）

```
 7      int x = 0;
 8      for (int i = 1; i <= n; i++) {
 9          cin >> x;
10      }
```

输出最后一个数之前，要先读完全部。可是每读一个，x 里的旧数就被换掉，读完只剩最后一个。

写 x1、x2、x3 三个变量可以存 3 个，可是 n 由输入决定，写多少个才够?

要有一种盒子，能一次准备一大排，还能用**编号**来找到其中一个。

# 数组：一排带编号的盒子

three.cpp

```
5       int a[3] = {4, 7, 2};
6       cout << a[0] << endl;
7       cout << a[1] << endl;
8       cout << a[2] << endl;
```

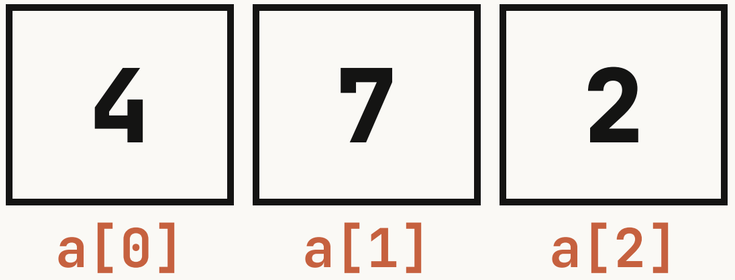
屏幕上

4
7
2




int a[3] 准备 3 个放整数的盒子，整排叫 a。
方括号里的数叫**下标**，a[1] 就是编号 1 的盒子。

# 下标从 0 开始




第 1 个盒子的编号是 0，第 2 个是 1，第 3 个是 2。
编号比“第几个”小 1。

3 个盒子的编号是 0、1、2，**没有 a[3]**。
C++ 的数组就是这样规定的：编号表示离第一个盒子有几步，第一个走 0 步。

# 小测：cout << a[2]; 输出几？

```
int a[3] = {4, 7, 2};
```

1 4

2 2

3 7

4 3

答对了！编号 2 是第 3 个盒子，里面是 2。这里编号和里面的数碰巧一样。

# 猜一猜：改掉一个盒子

change.cpp

```
5       int a[3] = {4, 7, 2};
6       a[1] = 9;
```

屏幕上

4

9

2




只有 a[1] 变了，旁边的盒子不受影响。

# 下标可以是变量，循环就能帮忙了

forward.cpp

```cpp
    int a[100] = {};
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }
    for (int i = 0; i < n; i++) {
        cout << a[i] << endl;
    }
```

= {} 让 100 个盒子都先放 0。

第一个循环存，第二个循环取。i 从 0 开始，到 n - 1 结束，所以写 i < n。

输入 3，再输入 4 7 2

4
7
2

n 是 1 到 100，每个数是 -1000 到 1000，都是整数。

# 输入 3，再输入 4 7 2：盒子一个个装满

| | | |
|---|---|---|
| i 是 0 | 0 < 3 成立 | 读到 4，放进 a[0] |
| i 是 1 | 1 < 3 成立 | 读到 7，放进 a[1] |
| i 是 2 | 2 < 3 成立 | 读到 2，放进 a[2] |
| i 是 3 | 3 < 3 不成立 | 循环结束 |

循环结束了，4、7、2 还都在盒子里。这就是数组比 x 强的地方：
第二个循环想从哪里开始取，就从哪里开始取。

# 猜一猜：输出时写成 i <= n

over.cpp（只阅读）

```
11      for (int i = 0; i <= n; i++) {
12          cout << a[i] << endl;
13      }
```

屏幕上

4
7
2
0

多出来一个 0。a[3] 有这个盒子，只是没读过数，还是 0。

**准备了 100 个，不等于存了 100 个。**n 才是真正存了几个。

# 小测：读入了 n 个数，最后一个是?

1 a[100]

2 a[n]

3 a[n - 1]

4 a[99]

答对了！第 1 个是 a[0]，第 n 个是 a[n - 1]。

# 动手改一改

从 forward 开始。读入不用动，只改第二个循环。

## 倒着输出

先读入 n（1 到 100），再读入 n 个整数（-1000 到 1000）。从最后一个输出到第一个，每个数占一行。

输入 3，再输入 4 7 2

```
2
7
4
```

输入 1，再输入 -5

```
-5
```

Dev-C++：打开 starter/03_reverse.cpp

# 动手改一改

也从 forward 开始。数组里的数不用动，只改输出的顺序。

## 第一个放到最后

读入同上。先输出第 2 个到第 n 个，最后输出第 1 个，每个数占一行。

输入 3，再输入 4 7 2

```
7
2
4
```

输入 1，再输入 -5

```
-5
```

Dev-C++：打开 starter/04_rotate.cpp

# 有了数组，就能同时看前后两个

diff.cpp

```
11      for (int i = 1; i < n; i++) {
12          cout << a[i] - a[i - 1] << endl;
13      }
```

输入 3，再输入 10 13 12

```
3
-1
```

一周的气温存进数组，今天比昨天变了多少？

a[i] 是这一天，a[i - 1] 是前一天。下标可以写算式。

n 是 1 到 100，气温是 -50 到 50，都是整数。

**i 从 1 开始**。第 1 天没有前一天，要是从 0 开始，就碰到 a[-1]，那个盒子不存在。

# 小测：输入 5 天气温，diff 输出几行?

1 5 行

2 4 行

3 3 行

4 6 行

答对了！i 是 1、2、3、4，一共 4 行，也就是 n - 1 行。

# 动手改一改

从 diff 开始。不用输出差，用第 9 课的计数。

## 有几天更热?

读入 n（1 到 100）和 n 天的气温（-50 到 50），都是整数。输出有几天比前一天更热，一样热不算。

输入 4，再输入 10 13 12 15

```
2
```

输入 3，再输入 5 5 5

```
0
```

Dev-C++：打开 starter/06_warmer.cpp

# 读报错：3 个盒子放 4 个数

three.cpp（只阅读）

```
5       int a[3] = {1, 2, 3, 4};
```

```
5:27: error: too many initializers for 'int [3]'
```

**too many** 是“太多了”：大括号里的数比盒子多。

方括号写 3，大括号里最多放 3 个数。少放可以，没放到的盒子是 0。多放不行。

Dev-C++ 显示的文字和位置可能不一样，还是先找行号和 error。

# 最危险的错：下标越界

out.cpp（只阅读，不要运行）

```
5       int a[3] = {4, 7, 2};
6       a[3] = 5;
```

编号只到 2，这里却用了 a[3]，写到了数组外面。

编译器可能给一个警告，也可能不说话，程序照样能运行。

结果说不准：有时看着没事，有时输出怪数，有时程序直接退出。

**写下标时自己算清楚：最小是 0，最大是 n - 1。**

# 一次输出整排？不行

whole.cpp（只阅读）

```
5       int a[3] = {4, 7, 2};
6       cout << a << endl;
```

屏幕上，每台电脑都可能不同

**0x16b98e578**

能运行，也没有 error，可是不会输出 4、7、2。这一串是盒子在内存里的位置。

想看里面的数，就用循环一个一个输出。

# 找错：哪一行有问题？

forward.cpp（只阅读）

```
1       for (int i = 0; i < n; i++) {
2           cin >> a[0];
3       }
4       for (int i = 0; i < n; i++) {
5           cout << a[i] << endl;
6       }
```

这段程序想原样输出。
输入 3，再输入 4 7 2，屏幕上是 2、0、0。
点你认为有问题的那一行。

点代码里的一行

# 自己挑战

# 高于平均分

先读入 n（1 到 100），再读入 n 个成绩（0 到 100），都是整数。输出有几个成绩高于平均分。

输入 5，再输入 60 70 80 90 100

**2**

平均分是 80，90 和 100 高于它

输入 3，再输入 80 80 80

**0**

先自己写，卡住了再翻下一页。

# 卡住了？一次看一条提示

**提示 1**　要读完全部才知道平均分，这时前面的数还在吗？所以要存进数组。

**提示 2**　第一个循环：读入，顺便累加总分。

**提示 3**　第二个循环：一个个和平均分比，符合就计数。小数的算法看第 9 课。

# # 造一个查询小程序

先把一串数存进数组，再让别人来查第几个。主题你自己定。

## 先做到

读入 `n` 和 `n` 个数，再读入 `k`，输出第 `k` 个。想想下标是几。

## 选你喜欢的

歌曲时长、座位号、零食价格、闯关得分，都可以。

## 还想更厉害

用 `while` 一直查，输入 `0` 才停；`k` 超出范围就提醒。

没有标准答案。写之前说清想要的结果，写完试试最小和最大的输入。

# # 和同桌换着玩

## 1

### 换着运行

运行同桌的程序，试试查第 1 个和最后一个

## 2

### 说一个喜欢的

告诉同桌，你最喜欢这个程序哪里

## 3

### 提一个想法

问问同桌：数都存下来了，还可以做什么？

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

# 说得清吗?

**Q1** **第 n 个数，下标是几?**

下标从 0 开始，第 n 个是 a[n - 1]。

**Q2** **int a[100] 里的 100 和 n 有什么不同?**

100 是准备了几个盒子，n 是真正存了几个。

**Q3** **什么题非要用数组?**

读完以后，还要回头用前面的数，比如倒着输出。

# 速查卡

```
int a[100] = {};  cin >> a[i];
```

| | |
|---|---|
| `a[0]` | 第 1 个，下标从 0 开始 |
| `a[n - 1]` | 读入 n 个时的最后一个 |
| `i < n` | i 从 0 走到 n - 1，正好 n 个 |
| `a[i - 1]` | 前一个，i 要从 1 开始 |
| **越界** | 没有报错也不能用，下标要自己算清楚 |

# # 这些后面再学

## 查找

某个数在不在数组里，在第几个

## 二维数组

一张表格，用行和列取数

## 排序

把数组里的数从小到大排好

今天学会数组，读过的数不再丢，想回头用哪个都可以。

# 谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

下一课：数组遍历与简单查找